Automating code generation from User Interface prototypes

João Paulo Castro
University of Minho & INESC TEC
Braga, Portugal
pg53929@alunos.uminho.pt

José Creissac Campos
University of Minho & INESC TEC
Braga, Portugal
0000-0001-9163-580X

***Abstract*—In web application development, the transition from design to the implementation phase is a time-consuming task. User-centred design is an iterative process that involves ideation, testing, and refinement to enhance usability, functionality, and the overall user experience. However, there is still a need to code the final design. In this paper, we propose an approach and its supporting tool to generate front-end code from a prototype of the user interface. Our goal is to generate code that can be easily integrated into the overall application development.**

***Index Terms*—Web Development Automation, front-end frameworks, Atomic Design, Component-driven prototyping**

I. Introduction

Web application development typically progresses through a number of phases, including analysis, prototyping, implementation, testing, and maintenance. Within this process, **User Interface** (UI) development represents a significant portion of the workload, be it during the initial creation of the product or subsequent improvements [1]–[3]. Frontend development can be particularly time-consuming due to the complexity of tasks such as handling user interactions, managing application state, and integrating with backend services.

Despite the availability of numerous frameworks, component libraries, and development tools designed to assist developers, UI development continues to constitute a substantial and essential part of the development effort. A core challenge is the dual perspective involved in building user interfaces: on one hand, designers approach the interface from a User-Centred Design (UCD) perspective, focusing on usability, design ideation, and validation with end users; on the other hand, developers focus on implementing the interface in a reliable and maintainable form. These two perspectives, although complementary in goal, operate with different assumptions and artefacts. UCD emphasises early-stage exploration, informal prototypes, and iterative feedback loops to ensure that the interface meets user needs. Developers focus on translating design into code, which can be time-consuming and error-prone, especially when requirements are constantly evolving.

One approach that has been proposed to help bridge the gap between design and implementation is Model-Based User Interface Development (MBUID) [4]. The goal is to reduce the manual work involved in UI development and improve consistency by transforming high-level abstract models that describe the UI into implementation code. However, because it relies on formal models rather than informal sketches or prototypes, MBUID does not easily fit into the iterative, user-driven nature of UCD, which hinders its adoption.

Advances in prototyping tools have narrowed the gap between UCD and implementation. Tools like Figma or Sketch support structured, machine-readable exports that go beyond static mockups. This allows prototypes to serve not only as communication artefacts but also as potential inputs for automated development workflows. In this paper, we propose a tool (FIGMA2VUEJS) that follows an MBUID approach to reduce the disconnect between UCD and implementation. By seeing user interface prototypes as formal interface models, our tool supports a transformation pipeline that bridges informal design with structured development. This allows developers to benefit from automation while preserving the flexibility and expressiveness valued in user-centred design processes.

II. Related work

Tools have been developed to ease the design and prototyping of user interfaces. Examples include Figma, Sketch, UXPin, Moqups, and others [5]. Amongst these options, Figma has gained popularity due to its capacities, such as real-time collaboration, cross-platform compatibility, and an extensive library of components [5]. Figma prototypes are organised into frames (also known as *artboards*). Design elements can be grouped into components, which may have variants to reflect different states or interactions. Transitions between frames can be defined to simulate user interactions. The tool supports plugins that enhance various aspects of the design workflow, such as layout, content, accessibility, and code generation, thereby reducing the gap between design and implementation.

One of the most popular code generation plugins is the Anima plugin [6], which uses machine learning to map design components to their equivalent code. Anima can generate projects for various frontend frameworks such as Vue.js and React, making it a useful starting point for representing the visual structure of the UI. However, the generated code primarily focuses on visual presentation and lacks functional implementation (i.e., behaviour logic). For instance, it lacks code for opening and closing overlay elements or managing the state of component variants. Furthermore, the generated UI is not fully responsive, and the code presents poor practices such as absolute positions. Other drawbacks include issues

related to interpreting the referential coordinates from Figma *artboards*. In addition, the effectiveness of these plugins is influenced by the organisation of the source code, the selected builder, and the version of the framework used in the project. The plugin integrates Generative AI features that enable developers to adjust the code in relation to the design, UI logic, or additional functionalities using natural language prompts [7]. However, these techniques may generate code that does not align with the designer's intended behaviour, potentially delaying the development process.

Besides Anima, many other LLM-based solutions have been proposed with the aim of converting prototypes to web apps. Two examples are Figma Make [8] and Vercel's V0 [9]. In these approaches, it is important to provide well-crafted and descriptive prompts in order to give context to the LLM. This helps the LLM correctly interpret which prototype elements correspond to pages in the web application or what behaviour is expected for navigation actions. This aspect represents a disadvantage since it implies replicating, in the textual prompts, information that is already in the prototypes. In cases where the programmer wants to adjust the generated code, obtain code related to page routing, or integrate UI libraries, it will be necessary to give that extra information. Regarding UI layout, this method is not entirely accurate, especially in the initial iterations, necessitating multiple rounds of refinement and increasingly detailed prompts to achieve acceptable results. Compared to the first approach of code generation through plugins, Vercel's V0, overall, shows lower code quality in terms of interaction logic code and UI layout.

In recent years, low-code and no-code development platforms have gained significant traction in the software industry [10]. These platforms aim to reduce the amount of hand-written code and abstract away low-level concerns such as database configuration, ORM mapping, and integration with external services. Typically, they provide visual development environments equipped with pre-built user interface components, allowing developers and designers to assemble applications by reusing these standardised elements across projects. This approach builds on component-oriented development, which offers several advantages. By relying on well-defined, reusable components, these platforms can automate aspects of UI design and source code generation while improving maintainability. Since component structures are often immutable and consistent, updates and modifications become easier to manage. One limitation, however, is the loss of design flexibility. Users are constrained to the components and layouts offered by the platform. This can become a barrier in projects that require a high degree of customisation or originality in the interface design. As a result, while low-code solutions are well-suited for the rapid development of standardised applications, they may not be ideal for projects with unique or complex UI requirements.

III. Proposed approach

Our approach aims to combine the flexibility of prototyping with the automated code generation of MBUID. The strategy

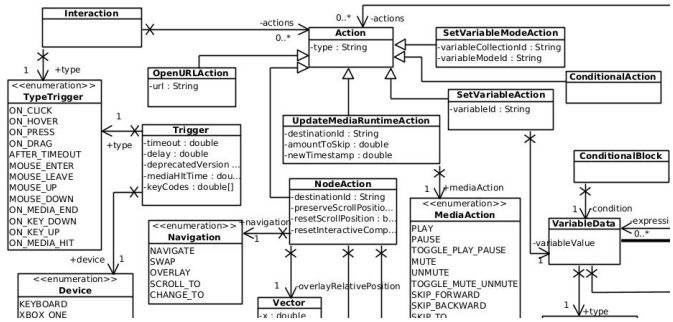

Fig. 1. Metamodel of Figma prototypes (Action & Interaction excerpt)

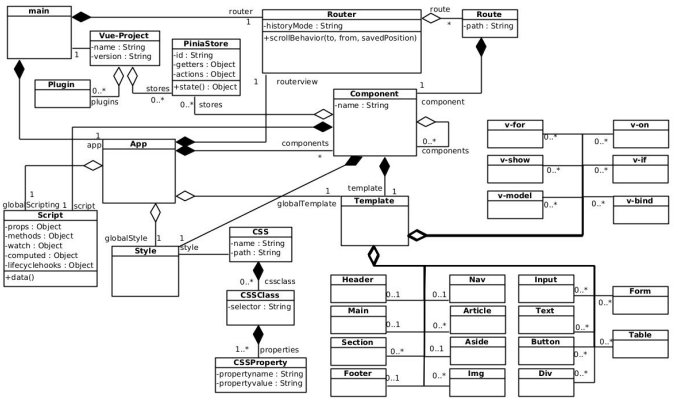

Fig. 2. Metamodel of a Vue.js project

to achieve this consists of seeing the prototype as the user interface model from which the code can be derived. This prototype-derived model is then progressively refined into a model of the implementation that supports code generation for a target UI framework.

Metamodels for the prototypes and the target UI framework are needed to support the refinement process. We will focus on Figma prototypes and the Vue.js framework (version 3). Obtaining information about the prototype is feasible through the Figma API, which makes it possible to extract information about the prototype and its nodes. A Figma prototype is characterised by different types of nodes (Frames, Sections, Instances, Components, Rectangles, etc.) with their associated styling (sizes, colours, strokes, effects, etc.), organised in a tree. The prototype also has information related to behaviour, such as user interactions and actions. Figure 1 presents an excerpt of the metamodel, focusing on interactions and actions.

A Vue.js project's component hierarchy is again organised in a tree, in which the App is the root and the components are the leaves. Each component will have a template section with a set of Vue directives, a script section and a style section. The project will also have a router which handles navigation between pages, a set of plugins, and a store to manage the application's state. Figure 2 presents the metamodel.

The approach leverages component-oriented prototyping based on the principles of Atomic Design [11]. Atomic Design makes an analogy between HTML elements and chemical

elements. It allows us to have a design system with elements that interact with each other to form components with well-defined structure, styling, behaviour, and logic. This will later facilitate the process of generating the corresponding code since elements in the design system are known in advance. The component-based approach seeks not only to simplify the design process but also the implementation phase. Generating the code of these components will be done using UI libraries such as Vuetify and PrimeVue, resulting in code with greater modularity and adaptability for future adjustments by developers. When elements are composed directly in the prototype to create new components, the analysis will make use of containment information relations (both structural, through the elements' containment hierarchy, and graphical) to identify and process them.

Regarding layout, the CSS grid system will be used. Through analysis of the position of the elements in the prototype, the best positioning of the elements in the grid will be determined.

IV. Implementation

In this section, we discuss the implementation of the proposed tool (FIGMA2VUEJS).

A. Architecture

As explained above, a model derived from a Figma prototype will be transformed into a Vue project model. For that, an intermediate model will be used that is built with information extracted from the prototype and structurally organised to link the two models.

Relevant entities in this intermediate model include: *Mpage*, *Melement*, *Mcomponent*, *InteractionElement* and *ElementAction*. All top-level frames in the prototype that have the "#Page" annotation in their name will correspond to an *Mpage* entity. This entity will then originate a Vue page with the relevant contents. The *Melement* entity might be one of *ContainerElement*, *ShapeElement*, *TextElement*, and *ImageElement*, and may contain other *Melements* nested within it, creating a tree structure, in the same way as elements can be nested in a prototype. This supports the representation of components defined directly in the prototype. If a design system is used, its components (which are predefined) are represented by subclasses of the *Mcomponent* entity. This organisation provides flexibility as both prototype-defined components and design system components can be represented. The latter can then be mapped to implementation components from Vue UI libraries (e.g., from Vuetify). The former will have to be analysed, and code will be generated to represent them.

The *InteractionElement* entity will correspond to the interactions that users can have with a given element of the prototype. Among others, this entity is characterised by a trigger action (e.g., *ONCLICK*, *ONHOVER*, *ONDRAG*) and a response, such as navigation, opening overlay elements, changing variant states, and scrolling events, among others. The *ElementAction* entity will be the superclass of trigger response actions.

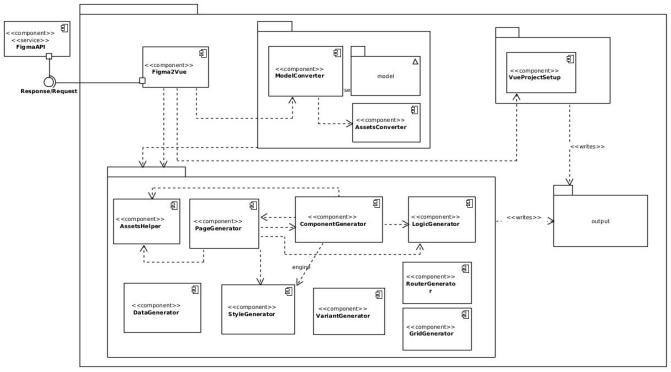

Fig. 3. Component Diagram (server-side)

Regarding the general architecture adopted for FIGMA2VUEJS, a division by architectural components was followed (Fig. 3), with a view of separating blocks by their responsibilities. The Figma2Vuejs component is the tool's entry point. This is where the data will be fetched from the selected prototype through the Figma API. At startup, it can receive information regarding the grid system to apply to the Vue pages. The ModelConverter component is where the data obtained from the Figma API will be parsed, and where the intermediate model described above will be built and stored. The AssetsConverter is where sizes, colours, texts, images and other properties will be extracted to build the components that will later be converted into Vue UI library components. The setup directory will contain a VueProjectSetup component that will create the Vue project with the name associated with the Figma prototype, and inject all the necessary dependencies and components from the design system present in the prototype.

The engine block (bottom left in Fig. 3) is where most of the generation and rule transformations will be done. The PageGenerator and ComponentGenerator components process and create the Vue pages and UI components, respectively. The StyleGenerator maps the elements' properties to CSS rules. The LogicGenerator is responsible for creating functions and JavaScript code associated with different Vue hooks, depending on the element's behaviour. The RouterGenerator generates code that is responsible for forwarding each route to a Vue page. The VariantGenerator generates dynamic components, which change their state whenever a user-triggered action is detected. Lastly, the DataGenerator is responsible for creating data objects to be displayed on the presentation layer.

B. Major Challenges

The generation of a Vue project from an intermediate model raises challenges that are discussed in this section.

1) Challenge 1 – Automating the setup of a Vue project: Setting up a Vue project involves multiple manual steps: initialising the project, configuring routing and state management, installing dependencies, and removing boilerplate, which can be time-consuming and error-prone when done repeatedly or at scale. The strategy followed consists of using

Python's *"subprocess"* module, which allows us to run bash or shell commands in the Python code itself, which are then executed by newly created processes. Functions were also created to manage and inject the UI library components in main.js, and to set a toast service store to explore the reactivity of elements such as Forms and Sliders when data is submitted. After obtaining the Vue project, additional processing is done to add global styling and to remove files with boilerplate code that isn't necessary. In this block, the installation of project dependencies is also automated by creating a shell script file with the npm install and run commands to execute the project.

2) Challenge 2 – Element positioning and grid system: Translating absolute positioning from Figma prototypes into a responsive and maintainable CSS grid layout poses a significant challenge due to differences in coordinate systems, nesting behaviour, and the need for layout normalisation. The strategy followed aims to provide the user with the option of choosing the desired division for the web pages. By default, each page is divided into a grid with 64 columns and 128 lines, where row height is dynamically calculated based on the page's height in the prototype to allow for vertical growth. Figma's coordinate system uses the upper-left corner of a layer's bounding box as its reference point, and therefore, these need to be normalised to use the page's reference and not the *artboard*'s. Firstly, we adjust every node to consider the coordinates of its parent node in the hierarchy as its reference point. Then, we perform a translation of each page to the point (0,0) and subtract that translation vector from all the elements that are contained at the first level of the page. Thus, based on the length and width of the page elements, it is possible to calculate the *grid-column-start*, *grid-column-end*, *grid-row-start* and *grid-row-end* properties for each element. Since the nested nodes will have an associated subgrid, the algorithm can be applied recursively, ensuring consistent layout behaviour across all hierarchy levels.

3) Challenge 3 – Transformation rules: Defining transformation rules that convert abstract design nodes into meaningful, semantically appropriate, HTML elements is a complex task, particularly when accounting for variations in node types, user intent, and interactivity. To manage this, the tree structure that represents each prototype's page is iterated over to apply the appropriate transformations for each node type. Each class representing a prototype node has an associated object corresponding to the node's style. One of the attributes of these classes is the 'tag'. If the designer adds the #tag annotation at the end of a node's name, its value is used to guide the transformation and defines the HTML tag to be generated. Otherwise, default mappings are applied: ImageElement objects will have the 'img' tag, TextElements will have the 'p' or 'a' tags (depending on whether they have a navigation action), ContainerElements and ShapeElements will have the 'div' tag. Additionally, each Figma node has a unique ID, which makes it easy to manage CSS selectors and assign rules.

4) Challenge 4 – Design system & Vue plugins: Integrating a custom design system with existing Vue UI libraries presents a challenge due to the need to accurately map abstract design components to concrete implementations while preserving intended behaviour and appearance. The strategy for converting components from the design system into functional components of PrimeVue and Vuetify begins by reserving the following names: *InputSearch*, *DatePicker*, *Dropdown*, *ReadOnlyRating*, *InteractiveRating*, *Paginator*, *Form*, *Checkbox*, *Menu*, *Slider* and *Table*. When these elements are identified in the prototype, a parsing step is performed over them, and relevant properties of each type of component are saved (e.g., list of items, colours, input text, etc.). Depending on the components detected, the algorithm maintains a dictionary of dependencies to update the content of the plugin files dynamically. The components from our design system are mapped to components from either PrimeVue or Vuetify, ensuring seamless integration between the custom design and the selected Vue UI libraries.

5) Challenge 5 – Extracting images and icons: Extracting image and icon assets from Figma prototypes introduces challenges related to network dependency, format consistency, and efficient batching of API requests. Since Figma files are stored on Amazon AWS, it is possible to select specific nodes from the prototype file and download them in a given format, such as .svg or .png. To manage this process, two global lists are initialised to store the IDs from nodes of vector type and of rectangle type, respectively. By storing these resources locally, generated Vue code becomes independent of external services, unlike the code obtained from the Anima plugin. Furthermore, the algorithm was adapted to make API calls in batches of size 50, in order to avoid large request payloads.

6) Challenge 6. – Overlay elements: Managing overlay elements in a prototype is challenging due to the need to accurately position overlays relative to trigger elements and to maintain correct hierarchical relationships within the page's DOM structure. From the element that triggers the action, it is possible to extract the offset coordinates (rx, ry) where the overlay should appear. For elements such as Frames or other first-level artboard nodes involved in hover interactions, it is necessary to check the location of the trigger element within the page's DOM hierarchy. Suppose the trigger element is contained within a node belonging to one of the pages. In that case, the algorithm dynamically inserts the overlay element into the trigger element's parent (the immediate hierarchical ancestor). In cases where the overlay overlaps the trigger element, the involved elements are assigned a hover property: the overlay starts with an opacity of 0, and transitions to opacity 1 on hover. At the same time, the trigger element transitions from opacity 1 to 0. If the overlay overlaps any component, the component's structure is recursively modified to include the overlay as a child of the relevant parent node. Furthermore, the algorithm builds the path from the trigger element to the overlay element, which is necessary for managing signal emissions in opening and closing actions.

7) Challenge 7 – Overlapping on shape elements: Shape nodes (geometric objects in Figma prototypes, such as *rectangles*, *ellipses*, *stars*, etc.) do not support nesting of child nodes, even though we can overlap elements on top of them

visually (depending on the order of layers in the prototype). If a node is within the spatial boundaries of a shape and appears after it in the layer list, the shape will visually overlap that node; if it appears before, the opposite occurs. The algorithm traverses the DOM structure of each page, and, for each pair of elements, if one is a shape node with a lower index in the elements list, and the grid boundaries of the other element are fully contained within it, the second element is added as a child of the *shape*, which is configured to have a grid display. This strategy is applied recursively to all child nodes, across every page of the prototype.

8) Challenge 8 – Variants conversion: Variants provide a way to define multiple configurations or states for a single component. The modelling strategy begins with a top-level dynamic component, which serves as a container for all variants. Each variant is converted into its own state component, representing a specific configuration. These state components are managed by the dynamic container, which uses a computed property to determine the current state based on user interaction. The logic for handling state changes involves passing the selected variant and the relevant selectors (keys or properties associated with each state component) as props. This enables the dynamic component to apply the appropriate properties and render the correct state. For each dynamic component, its state components and their corresponding selectors are stored.

9) Challenge 9 – Data generation: Extracting structured data from design elements is challenging due to the need to infer semantic meaning and group related fields without explicit data models. In order to represent a data entity, a "#data" tag must be appended to the names of the relevant nodes. When the algorithm finds a "#data" node, it performs a recursive parsing of the node to extract attribute–value pairs. Nodes in the same container will be part of the same data list. If a data entity is not enclosed in any container, the entity is added as an individual object in the page's data property. When generating a page or component, the ID of the corresponding node is needed to perform data interpolation. To achieve this, a dictionary is passed through props, allowing the entity's values to be dynamically interpolated into the component template.

10) Challenge 10 – Behavior's logic code generation: Mapping user interactions from Figma prototypes to appropriate Vue event handlers requires supporting a variety of interactions and automatically generating meaningful and maintainable logic. Depending on the interactions that a node presents in the Figma prototype (e.g., click, hover, drag), the corresponding Vue event handler (such as @click, @mouseenter or @mouseleave) can be associated with the relevant node. A dictionary data structure was created, where each key represents a Vue hook and maps to a list of method names and lines of JavaScript code that should be executed when the event is triggered. Different methods were created depending on the type of action. For example, in Navigation, the $router property is used, in Scroll, the scrollIntoView method is used, in Overlay, the path of the signal emissions is tracked, and the visibility of the elements involved is changed depending on the event, etc. Additionally, when working with well-defined components from UI libraries, additional behaviour can be automated. For instance, in the Form component, validation methods are created, the Slider component is configured to display a toast message showing the current value whenever it changes, for the Dropdown and Checkbox components, data is pre-populated within the mounted lifecycle hook, simplifying future integration with APIs, etc.

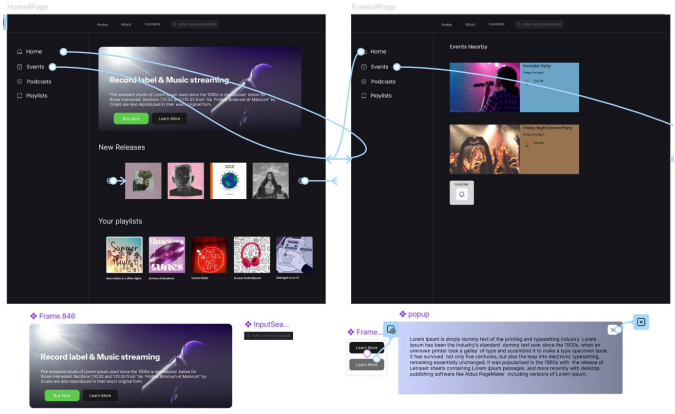

Fig. 4. Use case prototype made in Figma

V. Results

To validate the tool, a set of 36 prototypes (some purpose-built, others imported from open-source projects) was converted to code with both our tool and the Anima plugin. In the imported prototypes, minor modifications were made, including adaptations in the nodes' names and inclusion of components from our design system, to adjust them to our approach.

Fig.4 illustrates one of the prototypes used. This prototype, a generic music streaming platform, has two pages, one variant component, one generic component that contains the variant, 1 component from our design system (InputSearch), and one overlay component. The prototype features: page navigation; scroll and draggable elements; interaction with modal, input and button components; style representation, such as colour gradients, text fonts, sizes, shapes, images, etc.; reusable components; display of data; and state changes through variants.

A. Visual Fidelity

To analyse the outcomes (cf. Figures 5 and 6), metrics were calculated to determine any visual differences between the generated web UI's and the Figma prototypes. First, a pre-trained Convolutional Neural Network (CNN) based on the ResNet50 model was used to evaluate the cosine similarity and the Euclidean distance between the set of generated web UI's and the set of prototypes. When an image is passed through the CNN model layers, filters (matrices that were learnt from training) are applied to the image, which generates other images with different features detected from the original image. As each filter identifies and highlights a specific pattern from the image, it produces a feature map, in which the tensor values vary depending on whether a feature was detected more

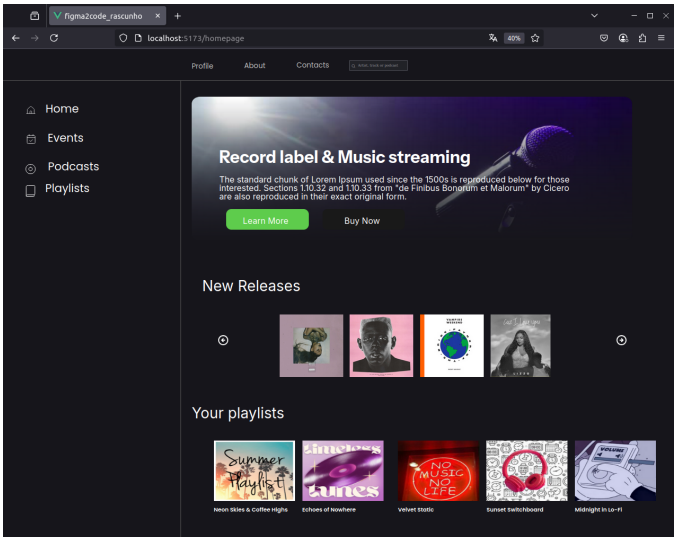

Fig. 5. Conversion of the use case of Fig. 4

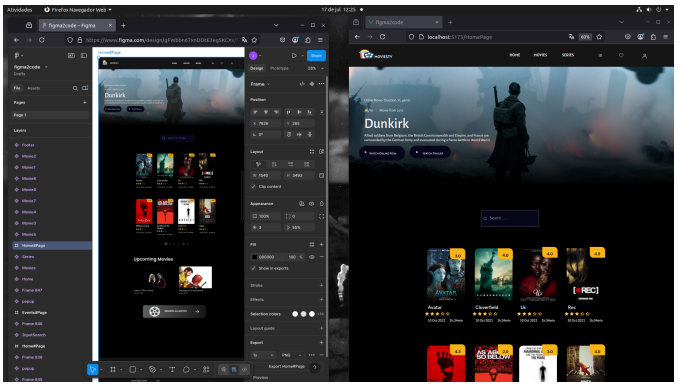

Fig. 6. A more complex prototype (left) and its generated web output (right)

in a region or not. We extracted these feature maps from the intermediate layers of the model to calculate the activation differences between the two inputs (images generated with our tool and images from the prototype) and visualise the low and high-level differences through heatmaps. This allowed us to identify which types of elements differ the most and which are identical or nearly identical between the two sets.

The other metric that was considered is related to the computation of the structural similarity index measure (SSIM). This metric is a method that seeks a perceptual comparison based on structural inter-dependencies between pixels [12]. We measured the SSIM between the web UI's and the prototypes using Python's module structural_similarity from skimage.metrics, and analysed how accurate the images are and where they diverge using the OpenCV library.

For this purpose, 34 images of web pages generated with our tool and their respective prototypes on Figma were exported to .png and .jpg formats. The ResNet50 model was loaded with the pre-trained weights of the ImageNet dataset. The top layer, which outputs the final classification, was removed, and the average layer pooling was selected to compute the average of the feature maps. The two sets were preprocessed in batches and passed to the model. The resulting embeddings were extracted, and the cosine similarity was calculated through the dot product of each row (that represents one of the images) of the normalised embeddings. In order to calculate the Euclidean distance, we used Python's numpy.linalg.norm module with axis=1 to get a list of distances for each pair of images.

For L2-normalised embeddings, the cosine similarity ranges from -1 (opposite direction) to 1 (same direction), and the Euclidean distance ranges from 2 (opposite) to 0 (identical).

TABLE I
COSINE SIMILARITY AND EUCLIDEAN DISTANCE METRICS

N=17 pairs	cosine similarity	euclidean distance
average	0.96	0.27
least similar	0.88 (pair 1)	0.49 (pair 1)
most similar	0.99 (pair 15)	0.17 (pair 15)

Overall, the generated web UI's and the respective visual Figma prototypes are very similar, resulting in an average cosine similarity of 0.96 and an average Euclidean distance of 0.27. Even though the most deviated pair scored 0.88 for cosine similarity and 0.49 for Euclidean distance, the differences are not too significant. In order to understand where they diverge, we built a secondary model which outputs some of the intermediate layers from the previous model. Three layers were selected: conv1_relu, conv2_block3_out, and conv3_block4_out, which capture low-level (eg, textures) to higher-level features (eg, structural layout). Next, the prototype embeddings and the generated UI embeddings were extracted from those selected intermediate layers, which resulted in 4D tensors of shape (batch size, height, width, channels). Finally, for each pair of corresponding feature maps, the absolute difference was computed, and the sum across all channels was computed at each spatial location. This produced a 2D heatmap in which each position indicates the magnitude of the feature difference between the pair of webUI and prototype.

The values for each spatial location on the images are represented with colours that vary in intensity, ranging from black, red, yellow and white (black represents regions almost identical, and white represents regions with more discrepancies). As seen in Fig. 7 (top), the lightest region coincides with a FORM component of our design system. This occurs since we are converting these types of components into blocks from which only key features are being extracted. In this specific case, we are extracting the number of inputs, placeholders, the form position, and colours from the input, text, and button. However, other features, such as spacing between inputs, the width and height of each input, text fonts, etc., are aspects that need to be taken into consideration when overwriting the styling of these components from UI libraries. Meanwhile, in Fig.7 (bottom), the lighter regions correspond to shape elements such as rectangles, ellipses, etc., that are being implemented with the clip-path property, and sometimes the elements contained are being cropped and a bit unformatted due to the grid-system allocation.

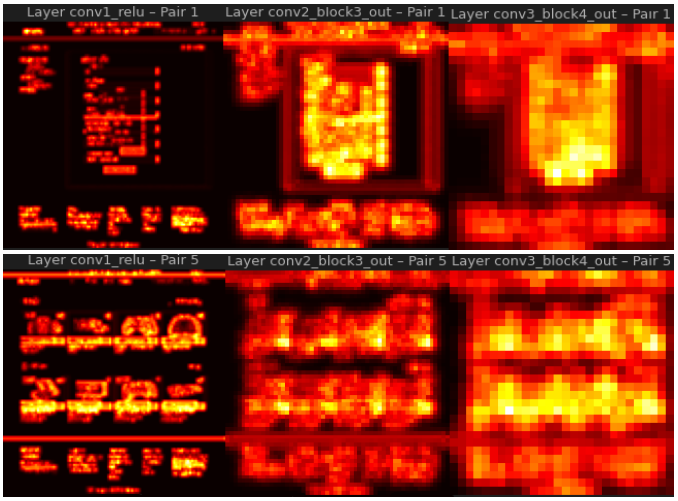

Fig. 7. Feature map differences in 3 different layers of the model: pair 1 (top), pair 5 (bottom)

TABLE II
COSINE SIMILARITY, EUCLIDEAN DISTANCE AND SSIM

	Anima	**FIGMA2VUEJS**
cosine similarity	0.97	0.96
euclidean distance	0.23	0.27
ssim	0.79	0.81

On average, Anima performed slightly better at maintaining geometric fidelity when converting prototypes to Vue code. In some cases, layouts were significantly impacted by Anima's approach to implementing component states and overlay elements. In other cases, pages were not even created and therefore did not impact the above metrics. Although the vector similarity is slightly lower, our tool has the advantage of representing the UI in a grid system, which is a good design practice for screen adaptability. In Anima, as that approach was not followed, we had to adjust the size of the generated page to the screen in order to screenshot the viewport and calculate the metrics; otherwise, that would have an impact on the metrics. However, measuring the SSIM, which ranges from -1 to 1 (1 indicating perfect similarity between two images and negative values indicating dissimilarity), our tool behaves slightly better. This metric indicates how similar the prototype image and the respective web UI image are from a human visual point of view (based on contrast, luminance and structure), resulting in a score of 0.81 for our tool and 0.79 for Anima.

Although our tool achieves a slightly higher SSIM score (0.81 compared to Anima's 0.79), the difference is relatively minor. Given the perceptual nature of SSIM and the small absolute difference, the two approaches can be considered approximately equivalent in terms of visual fidelity from a human point of view.

B. Code Analysis

As a method of validation, the Vue code generated from our tool was evaluated using SonarQube. For this purpose, the 36 projects that were generated from the prototypes were considered. The evaluation of the code quality was based on the following metrics: security, reliability, maintainability, size & comments, and complexity.

A complete code review found no security vulnerabilities or potentially dangerous patterns. Among the 36 projects, the code holds an A reliability rating, though 25 Medium-level issues remain open. These issues are related to the fact that SonarQube, contrary to native elements, does not detect components imported from Vue UI libraries. Therefore, a lack of matching control on input elements was reported, even though an ID was associated with the component used to implement them (InputText from the PrimeVue library). In the maintainability metric, an A rating was achieved, with 72 code smells detected across the 36 Vue projects (average of 2 per project). The total technical debt is approximately 2h52min, with all issues classified as low complexity. These issues are related to duplicated CSS selectors in variant components and duplicated CSS selectors from component instances.

Regarding the size & comments metric, the 36 projects contained 178,572 lines of code (LOC), 1255 files, 1423 functions and 171 comment lines. As the current amount of comment lines only represents 0.1% of the overall code, further effort is needed to generate docstrings in more complex functions, provide additional comments indicating component adaptations, and explain potential code adjustments.

As a starting point for a more nuanced comparison between the quality of code generated through our approach and that of Anima, a more complex prototype, based on a movies application, will be taken into account. The prototype is shown in Fig. 6.

TABLE III
CODE SIZE, COMPLEXITY AND DUPLICATION

Metric	**Anima**	**FIGMA2VUEJS**
Lines of Code (LOC)	3,924	10,081
LOC (no duplication)	3,924	9,772
Functions	1	132
Cyclomatic Complexity	2	143
Duplication Density	0.0%	2.2%
Duplicated Lines	0	309
Duplicated Blocks	0	28
Comment Lines	1	6

As illustrated in Table III, the Anima approach generates cleaner code with no duplicated lines or blocks of code, resulting in 0% of duplication density, compared to the 2% in our tool. This indicates a need for further refinement in order to reduce code redundancy and produce higher maintainability. Even though the Vue project generated with FIGMA2VUEJS has more duplications, it presents a higher number of functions (132 vs 1 function) and a higher number of lines without duplication (9,772 against 3,924 lines). Furthermore, the code produced by our tool achieved a higher score of 143 at cyclomatic complexity, a metric that quantifies the number of linearly independent paths through a program's source code. However, this is the result of functionally richer and scalable code, as demonstrated in Table IV.

TABLE IV
CODE FEATURE COMPARISON - ANIMA VS FIGMA2VUEJS

Code Feature	FIGMA2VUEJS	Anima
Page navigation	✓(scripts)	✓(templates)
Props	✓(only #data components)	✓
Signal emissions	✓(overlay open & close)	×
Conditional rendering	✓	×
Vue hooks	✓	×
Event handlers	✓	×
Data binding	✓(one & two-way)	✓(one-way)
Loop rendering (v-for)	✓	×
Dynamic components	✓(variant components)	×
Validation Methods	✓(form component)	×

TABLE V
SECURITY, RELIABILITY, AND MAINTAINABILITY

Metric	Anima	FIGMA2VUEJS
Security Rating	A	A
Bugs (Reliability)	1	0
Reliability Rating	C	A
Code Smells	3	0
Technical Debt	15 min	0 min
Maintainability Rating	A	A

Even though our approach has slightly more comment lines than Anima (ratio of 0.1% vs 0.0%), it still needs to be further developed to give more insightful documented code so that programmers can know how to adapt the code, how specific components were converted and how they work, etc.

In terms of security, no vulnerabilities or insecure code patterns were found on the Vue projects, receiving the highest score of A. With regard to the reliability metric, the Anima code shows one minor bug related to the lack of the lang attribute on the HTML element. This is a medium-category issue that requires a short time of 2 minutes. Other issues with the Anima code were related to the redundant alt attribute values in img elements, involving a technical debt of 15 minutes. Meanwhile, on our approach, no code smells were detected, and there was no technical debt, which results in good codebase maintainability.

VI. CONCLUSION

Despite Anima's approach being simple and presenting a cleaner codebase, it shows a lack of functional behaviour in the code. This solution, which uses Vue2 standards, is more suitable for smaller-scale projects with little associated logic and complexity. However, both the Anima plugin and our tool proved to be good solutions to achieve overall visual fidelity as evidenced by metrics such as cosine similarity, Euclidean distance and SSIM.

Our approach, which applies current Vue3 standards and architecture with the Options API, is more favourable for large-scale projects and professional maintenance. It also supports modular reactivity, state management with Pinia, and a more advanced logical composition. Additionally, our tool ensures a more precise separation between components and views by generating component files only for nodes of type "component" on the *artboard*, unlike Anima, which places all element nodes in the components folder, reducing developer control. Furthermore, Anima's strategy, based on a centralised JSON data file passed via props, may ease API integration but could affect performance when components primarily handle static data. The choice between the two approaches depends on the desired maintenance effort and level of automation from the design to the implementation phase.

In order to improve our tool, some future work can be done to reduce duplicated code and to improve the visual accuracy between the prototypes and the generated web UIs. In order to reduce the visual discrepancies, it will be necessary to mitigate the approximation errors in the grid system allocation. Even though a strategy was followed to solve challenge 7, some visual accuracy was affected by the shape elements and the fact that they do not support nesting. The issue is reduced when the prototype is built using a strictly component-based prototyping approach. Still, as we aimed to give design flexibility, we provide a solution for when shape nodes are used and visually combined with other elements. Additionally, there is an opportunity to extend the tool to integrate more new components into our design system. At the code level, we could adapt our tool to generate code following the Composition API, making it more flexible for developers to choose whatever standard they prefer.

REFERENCES

[1] B. A. Myers and M. B. Rosson, "Survey on user interface programming," in *Proceedings of the SIGCHI conference on Human factors in computing systems*, 1992, pp. 195–202.

[2] F. Daniel, M. Matera, J. Yu, B. Benatallah, R. Saint-Paul, and F. Casati, "Understanding ui integration: A survey of problems, technologies, and opportunities," *IEEE Internet Computing*, vol. 11, no. 3, pp. 59–66, 2007.

[3] R. Kennard and J. Leaney, "Towards a general purpose architecture for ui generation," *Journal of Systems and Software*, vol. 83, no. 10, pp. 1896–1906, 2010.

[4] E. Schlungbaum, "Model-based user interface software tools - current state of declarative models," Graphics, Visualization & Usability Center, Georgia Institute of Technology, Tech. Rep. GIT-GVU-96-30, 1996.

[5] M. Soegaard, "8 best prototyping tools for UX designers in 2025," Interaction Design Foundation - IxDF. https://www.interaction-design.org/literature/article/best-prototyping-tools (accessed July 16, 2025), February 2025.

[6] A. Cohen, "Introducing Anima Playground and Anima API: UI-first AI code generation," https://www.animaapp.com/blog/product-updates/introducing-anima-playground-and-anima-api-ui-first-ai-code-generation/ (accessed May 12, 2025), March 2025.

[7] ——, "Introducing Generative AI in design-to-code at Anima," https://www.animaapp.com/blog/industry/introducing-generative-ai-in-design-to-code-at-anima/ (accessed May 12, 2025), October 2023.

[8] P. Ng, R. Chouhan, and T. Duncalf, "Introducing Figma Make: A new way to test, edit, and prompt designs," Shortcut. https://www.figma.com/blog/introducing-figma-make/ (accessed June 12, 2025), May 2025.

[9] I. Udoidiok, H. Reza, and J. Zhang, "Exploring ai integration in software development: Case studies and insights," in *NAECON 2024 - IEEE National Aerospace and Electronics Conference*, 2024, pp. 375–380.

[10] H. Shah, "Gartner forecast on low code development technologies in 2025," ToolJet. https://blog.tooljet.ai/gartner-forecast-on-low-code-development-technologies/ (accessed May 12, 2025), September 2024.

[11] B. Frost, *Atomic Design*. Brad Frost Web, 2016, available: https://bradfrost.com/blog/post/atomic-web-design/ (accessed May 24, 2025).

[12] T. Calò and L. De Russis, "Advancing code generation from visual designs through transformer-based architectures and specialized datasets," *Proc. ACM Hum.-Comput. Interact.*, vol. 9, no. 4, Jun. 2025.